Design and Implementation of a Sandboxed Tool-Augmented Large Language Model Agent

Si Thu San

Abstract
Large Language Models (LLMs) have demonstrated strong reasoning and natural language capabilities but lack intrinsic state, execution ability, and environmental awareness. This work presents the design and implementation of a sandboxed, tool-augmented agent architecture in which an LLM functions solely as a decision-making component, while all execution, memory persistence, and security controls are handled externally. The proposed system employs explicit function invocation, bounded execution loops, and filesystem sandboxing to safely enable iterative task completion. This architecture emphasizes determinism, auditability, and security over autonomous behavior, offering a practical foundation for controlled LLM-driven automation.

1. Introduction
Recent advancements in LLMs have enabled systems capable of multi-step reasoning and task decomposition. However, naive integration of LLMs with execution environments introduces significant risks, including uncontrolled code execution, data leakage, and hallucinated actions. This paper explores a constrained agent architecture that explicitly separates decision-making from execution, enforcing capability boundaries through user-defined tools and filesystem sandboxing.

2. Background and Motivation
LLMs are inherently stateless and lack the ability to interact with external systems without mediation. Existing agent frameworks often obscure execution boundaries or rely on prompt-based safety mechanisms. This work adopts a stricter approach, treating the LLM as an untrusted planner whose outputs must be interpreted and executed by a controlled runtime environment.

3. System Architecture
The system consists of four primary components:
• LLM Planner – A stateless LLM responsible for selecting the next action based on provided context.
• Agent Runtime – A Python-based control loop that manages iteration, context construction, and termination conditions.
• Tool Interface – A fixed set of user-defined functions enabling file inspection, file modification, and program execution.
• Sandboxed Execution Environment – A restricted working directory enforced via path normalization and validation to prevent unauthorized filesystem access.

4. Agent Control Loop
The agent operates using a bounded iterative loop: load persistent external state, construct context, invoke the LLM, execute the selected action within the sandbox, persist execution results, and repeat until termination. This structure resembles ReAct-style reasoning while maintaining strict separation between reasoning and execution.

5. Memory and State Management
The LLM maintains no internal memory across invocations. All state is stored externally and selectively replayed in subsequent prompts. This approach ensures transparency and reproducibility while avoiding implicit or uncontrolled memory accumulation.

6. Security Considerations
Security is enforced through capability restriction via explicit tool exposure, filesystem sandboxing using path validation, bounded execution loops, and absence of direct LLM access to system resources. These mechanisms prevent unauthorized access and mitigate risks associated with LLM

hallucinations.

7. Limitations
The system does not implement long-term learning, global planning graphs, or model self-evaluation. Memory replay is linear and may become inefficient for large histories. Future work may explore memory summarization and hierarchical planning.

8. Conclusion
This work demonstrates that practical LLM-based agents can be constructed using simple, explicit mechanisms without reliance on opaque autonomy. By treating the LLM as a planner rather than an actor, the system achieves controlled automation while preserving safety and determinism.